

Guia de Estudo — Exame Teórico de Bases de Dados 2025/2026

Formato do exame: 8 perguntas de desenvolvimento (sem consulta, 2h00m) **Cotação típica:** Perguntas de 2 valores (teóricas) + Pergunta de normalização (3 val.) + Pergunta prática SQL/Álgebra Relacional (5 val.) **Baseado em:** Exame Época Normal 2024/2025 + BD-Todas-As-Perguntas + Slides PPS

Índice

1. [Conceitos Fundamentais de BD](#)
2. [Arquitetura ANSI/SPARC e Independência de Dados](#)
3. [Modelo Relacional](#)
4. [Álgebra Relacional](#)
5. [SQL – LMD \(Linguagem de Manipulação de Dados\)](#)
6. [SQL – LDD \(Linguagem de Definição de Dados\)](#)
7. [Integridade Referencial](#)
8. [Vistas \(Views\)](#)
9. [Triggers, Stored Procedures e Funções](#)
10. [Normalização](#)
11. [Desenho e Modelação de BD \(Diagramas E/R\)](#)
12. [Data Warehousing](#)
13. [BD Distribuídas e Paralelas](#)
14. [Exercícios Tipo Exame \(Ver documento separado\)](#)

1. Conceitos Fundamentais de BD

Pergunta típica: "Defina BD, SGBD e Metadados"

BASE DE DADOS (BD): Coleção partilhada de dados logicamente relacionados e a descrição desses dados, desenhada para satisfazer a necessidade de informação de uma organização.

SISTEMA DE GESTÃO DE BASE DE DADOS (SGBD): Sistema de software que permite aos utilizadores definir, criar, manter e controlar o acesso à base de dados.

5 Componentes de um SGBD:

Componente	Descrição
------------	-----------

Hardware	Parte física — servidores, discos, rede
Software	Sistema operativo + SGBD + aplicações
Dados	A informação armazenada na BD
Procedimentos	Regras de utilização e interação com a BD
Pessoas	Utilizadores finais, administradores, programadores

METADADOS: Repositório de informação que descreve os dados na BD ("dados sobre dados"). Disponibiliza a descrição dos dados para obter aplicações independentes.

? "Diferenças entre Sistemas de Ficheiros e Sistemas de BD"

Aspeto	Sistema de Ficheiros	Sistema de BD
Dados	Separados e isolados	Centralizados e partilhados
Redundância	Elevada	Controlada
Acesso simultâneo	Limitado	Multiutilizador
Independência dados/aplicações	Não existe	Existe
Custo	Baixo	Elevado
Complexidade	Baixa	Elevada
Impacto em falha	Menor	Maior

Quando preferir Sistemas de Ficheiros: Quando a quantidade de informação é baixa, serve apenas um departamento, custo inferior, e o impacto em caso de falha é reduzido.

Vantagens do SGBD

- Controlo sobre a redundância de dados
- Consistência e integridade dos dados
- Partilha de dados
- Mais segurança
- Economia de escala
- Serviços de backup e recuperação
- Independência de dados

Desvantagens do SGBD

- Complexidade e tamanho
- Custo elevado (SGBD + hardware + conversão)
- Performance pode ser afetada
- Maior impacto em caso de falha

? "Sublinguagens de dados (DDL vs DML)"

DDL (Data Definition Language): Permite implementar a BD — criar tabelas, relações, restrições de integridade.

- Comandos: CREATE, ALTER, DROP

DML (Data Manipulation Language): Usada após a criação da BD — inserir, eliminar, atualizar e consultar dados.

- Comandos: SELECT, INSERT, UPDATE, DELETE

? "Diferenças entre LMD Procedimentais e Não-Procedimentais" ★ (Pergunta 1 do exame 2024/2025)

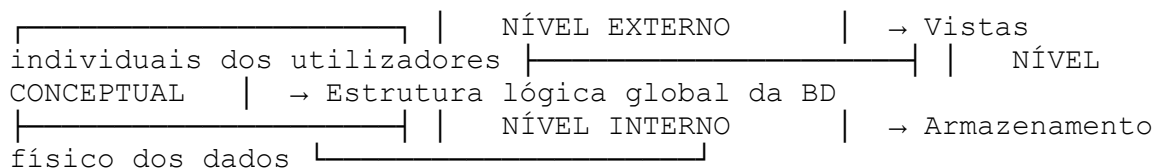
LMD Procedimental	LMD Não Procedimental
Especifica como obter os dados	Especifica que dados obter
Manipula registos um de cada vez	Opera sobre conjuntos de dados
Ex: linguagens de 3ª geração (C, Java + cursores SQL)	Ex: SQL

Exemplos:

- **Procedimental:** Cursores SQL (FETCH, OPEN, CLOSE), linguagens hospedeiras
- **Não-Procedimental:** SQL puro (SELECT ... FROM ... WHERE)

2. Arquitetura ANSI/SPARC e Independência de Dados

? "Identifique os três níveis da arquitetura ANSI/SPARC"



Nível	O que representa
Externo	Visão de um utilizador sobre a BD; descreve a parte relevante para esse utilizador
Conceptual	União das vistas; descreve entidades, atributos, relacionamentos, restrições, segurança
Interno	Representação física — como os dados são armazenados no computador

Objetivos da Arquitetura ANSI/SPARC

- Todos os utilizadores acedem aos **mesmos dados** sem conhecer detalhes físicos
- Cada utilizador tem uma **vista imune** a alterações noutras vistas
- O administrador pode **alterar a estrutura de armazenamento** sem afetar utilizadores
- A estrutura interna não é afetada por alterações no armazenamento físico

? "Independência de Dados"

Mudanças nos níveis inferiores (lógico ou físico) **não afetam** os níveis superiores.

? "Arquitetura Cliente-Servidor 2 vs 3 níveis"

2 Camadas	3 Camadas
Servidor BD + Cliente (aplicação)	Servidor BD + Servidor Aplicação + Interface
Menos escalável	Suporta balanceamento de carga
Menos adequada para web	Mais adequada para web

3. Modelo Relacional

? "Explique: Relação, Atributo, Domínio, Tuplo, Grau, Cardinalidade"

Termo	Definição
Relação	Tabela com colunas e linhas (estrutura lógica)
Atributo	Nome de uma coluna de uma relação
Domínio	Conjunto de valores permitidos para um ou mais atributos

Tuplo	Uma linha de uma relação
Grau	Número de atributos de uma relação
Cardinalidade	Número de tuplos de uma relação

Propriedades das Relações

- Nome **único** no schema
- Cada atributo tem um nome **distinto**
- Valores pertencem ao **mesmo domínio**
- A ordem dos atributos **não tem significado**
- Cada tuplo é **diferente** (não há duplicados)
- A ordem dos tuplos **teoricamente** não interessa (na prática afeta performance)

? "Regras de Integridade do Modelo Relacional"

1. **Integridade da Entidade:** Nenhum tuplo pode ter valor NULL na **chave primária**
2. **Integridade Referencial:** Se um atributo é chave estrangeira, só pode assumir valores que existam na tabela onde é chave primária (ou NULL)
3. **Integridade Geral:** Restrições relativas ao negócio da BD

? "Chave Candidata, Chave Primária, Chave Estrangeira"

Tipo	Definição
Chave Candidata	Um ou mais atributos que identificam unicamente um tuplo
Chave Primária (PK)	A chave candidata escolhida para identificação dos tuplos
Chave Estrangeira (FK)	Atributo numa relação que é PK/CK noutra relação; permite relacionar tuplos

3 Gerações de SGBD

1. **Modelo Hierárquico** — pouca independência de dados, programas complexos
2. **Modelo Relacional** — resolve muitos problemas da 1ª geração
3. **SGBD OO e Objeto-Relacional** — novo paradigma

4. Álgebra Relacional

? "Defina as 5 operações básicas de álgebra relacional"

Operação	Símbolo	Descrição
Seleção	σ	Seleciona tuplos que satisfaçam a condição
Projeção	π	Projeta as colunas solicitadas
Produto Cartesiano	$\times$	Combina tuplos de duas relações
União	$\cup$	Une duas tabelas (Union Compatible)
Diferença	$-$	Linhas de A que não estão em B

Operações derivadas

Operação	Definição
Junção ($\bowtie$)	Produto cartesiano + seleção
Intersecção ($\cap$)	Linhas comuns a A e B $\rightarrow A - (A - B)$
Divisão ($\div$)	Valores de A que referenciam TODOS os valores de B

? "Tipos de Junção"

Tipo	Quando usar
Theta Join	Atributos de junção NÃO são homónimos
Equijoin	Condição contém apenas igualdade (=)
Natural Join	Junção automática por atributos com mesmo nome
Outer Join	Mostra TODA a informação de uma relação (inclui sem correspondência)
Semijoin	Só aparecem tuplos de UMA relação que participam na junção

? "Como combinar resultados de duas queries?"

Através de UNION, INTERSECT, EXCEPT — as relações têm de ser **Union Compatible** (mesmo número de atributos, mesmos domínios correspondentes).

5. SQL – LMD (Linguagem de Manipulação de Dados)

? "Explique cada cláusula do comando SELECT"

```
SELECT  atributos          -- OBRIGATÓRIO: colunas que
aparecerão FROM  tabela          -- OBRIGATÓRIO: tabela de
origem WHERE  condição          -- Filtra LINHAS (não aceita
funções de agregação) GROUP BY atributo          -- Agrupa por
atributo HAVING  condição_grupo  -- Filtra GRUPOS (aceita
funções de agregação) ORDER BY atributo [ASC|DESC] -- Ordenação
(última cláusula, por defeito ASC)
```

? "Diferença entre WHERE e HAVING"

WHERE	HAVING
Filtra linhas individuais	Filtra grupos
Aplicada antes do GROUP BY	Aplicada depois do GROUP BY
Não aceita funções de agregação	Aceita funções de agregação

? "Importância do WHERE em UPDATE e DELETE"

Sem WHERE, o UPDATE e DELETE aplicam-se a **TODOS os registos** da tabela — o WHERE permite seleccionar apenas os registos específicos.

Funções de Agregação

Função	Descrição	Restrição
COUNT()	Conta registos	Única que conta NULLs (com *))

SUM()	Soma valores	Apenas numéricos
AVG()	Média	Apenas numéricos
MIN()	Valor mínimo	Qualquer tipo
MAX()	Valor máximo	Qualquer tipo

Regras:

- Se há função de agregação no SELECT sem GROUP BY, não pode haver referência a outra coluna (exceto a usada na função)
- Todas as colunas no SELECT que não estão em funções de agregação devem estar no GROUP BY
- Para eliminar duplicados: DISTINCT
- Exceto COUNT(*), todas ignoram valores NULL

? "Subquery vs Junção"

Subquery	Junção
Query embebida noutra query	União entre várias tabelas
Não pode ser usada como operador numa expressão	Mais flexível

3 tipos de Subqueries:

1. **Escalar** — devolve um valor singular
2. **De Linha** — devolve apenas um tuplo
3. **De Tabela** — retorna uma relação

6. SQL – LDD (Linguagem de Definição de Dados)

Comandos Principais

```
-- CRIAR TABELA CREATE TABLE NomeTabela (      coluna1 tipo [NOT
NULL] [UNIQUE] [DEFAULT valor] [CHECK (condição)],      PRIMARY
KEY (coluna1),      FOREIGN KEY (coluna2) REFERENCES TabelaPai
[ON UPDATE ação] [ON DELETE ação] ); -- ALTERAR TABELA ALTER
TABLE NomeTabela      ADD coluna tipo;      DROP COLUMN coluna;
ALTER COLUMN coluna SET DEFAULT valor; -- REMOVER TABELA DROP
TABLE NomeTabela [RESTRICT | CASCADE]; -- CRIAR ÍNDICE CREATE
[UNIQUE] INDEX NomeIndice ON Tabela (coluna [ASC|DESC]); --
REMOVER ÍNDICE DROP INDEX NomeIndice;
```


Restrições de Integridade em SQL

- **NOT NULL** — dados obrigatórios
- **CHECK** — restrições de domínio
- **PRIMARY KEY** — integridade de entidade (apenas uma por tabela)
- **UNIQUE** — garante valores únicos (chaves alternativas)
- **FOREIGN KEY REFERENCES** — integridade referencial

7. Integridade Referencial

? "Explique Integridade Referencial e ações ON DELETE/ON UPDATE" ★ (Pergunta 3 do exame 2024/2025)

Integridade Referencial preserva as relações definidas entre tabelas quando linhas são criadas ou excluídas.

Ações nas subcláusulas ON DELETE e ON UPDATE:

Ação	Comportamento
CASCADE	Apaga/atualiza a linha pai e as linhas correspondentes nas tabelas filhas, em cascata
SET NULL	Apaga/atualiza na tabela pai e muda as FK nas tabelas filhas para NULL (FK não pode ser NOT NULL)
SET DEFAULT	Apaga/atualiza na tabela pai e muda as FK para o valor DEFAULT especificado
NO ACTION	Rejeita a operação na tabela pai (comportamento por defeito)

```
-- Exemplo prático: FOREIGN KEY (staffNo) REFERENCES Staff ON  
DELETE SET NULL ON UPDATE CASCADE
```

8. Vistas (Views)

? "O que é uma vista? Diferenças entre vista e relação base" ★
(Pergunta 2 do exame 2024/2025)

Vista	Relação Base
Relação virtual	Existe fisicamente na BD
Pode não existir fisicamente	Armazenada em disco
Produzida em tempo real	Dados permanentes
Definida por uma query SELECT	Criada com CREATE TABLE

Vantagens das Vistas

- **Segurança** — esconde partes da BD de certos utilizadores
- **Simplicidade** — simplifica operações complexas
- **Personalização** — acesso à informação de forma personalizada

Desvantagens das Vistas

- **Restrições nas atualizações** — pode não ser atualizável
- **Restrições na estrutura** — definida na criação; alterações requerem nova vista
- **Performance** — se envolve junção de várias tabelas, a junção é feita sempre que é acedida

Condições para uma Vista ser Atualizável

- Sem GROUP BY ou HAVING
- FROM apenas refere **uma** tabela
- Sem DISTINCT
- Sem funções de agregação nem subqueries

? "Materialização de Vistas"

Consiste em armazenar a vista numa **tabela temporária** na BD, tornando o acesso muito mais rápido (melhor performance quando consultada frequentemente).

Mecanismo de Resolução de Vistas

1. Nomes das colunas da vista → traduzidos para nomes da definição
2. Nomes das vistas no FROM → substituídos pelas tabelas da definição
3. WHERE da query do utilizador → combinado com WHERE da definição (AND)
4. GROUP BY e HAVING → copiados da definição
5. ORDER BY → copiado da query com nomes traduzidos
6. Query final executada

```
-- Exemplo de criação de vista: CREATE VIEW Manager3Staff AS  
SELECT * FROM Staff WHERE branchNo = 'B003';
```

9. Triggers, Stored Procedures e Funções

? "O que são Triggers e para que servem? Vantagens e desvantagens?" ★ (Pergunta 4 do exame 2024/2025)

TRIGGER: "Dispara" uma ação ou conjunto de ações quando um evento ocorre na BD (INSERT, UPDATE ou DELETE).

Tipos de Triggers

Tipo	Quando executa
BEFORE	Antes da operação (INSERT/UPDATE/DELETE)
AFTER	Depois da operação
INSTEAD OF	Em vez da operação SQL "normal"

Vantagens

- Eliminação de código redundante
- Melhoria na integridade da informação
- Facilidade na alteração das regras de negócio
- Boa integração com a arquitetura cliente-servidor

Desvantagens

- Overhead do processador

- Possível efeito cascata
- Impossibilidade de agendar triggers
- Diminuição da portabilidade (cada SGBD tem sintaxe diferente)

```
-- Exemplo de Trigger T-SQL: CREATE TRIGGER [EmployeeUpdateAudit]
ON [Employee] AFTER UPDATE AS BEGIN      SET NOCOUNT ON;
INSERT INTO AuditEmployeeTable      SELECT i.EmployeeID,
GETDATE(), SUSER_SNAME(), d.JobRole, i.JobRole      FROM inserted
i JOIN deleted d ON i.EmployeeID = d.EmployeeID      WHERE
d.JobRole != i.JobRole END
```

Nota: A tabela deleted armazena cópias das linhas afetadas durante DELETE e UPDATE. A tabela inserted armazena cópias durante INSERT e UPDATE.

? "Diferença entre Procedimento e Função"

Procedimento (Stored Procedure)	Função
Não retorna valor no final	Retorna sempre um valor
Pode modificar dados	Não pode fazer modificações
Executado com EXEC	Usado dentro de SELECT

```
-- Stored Procedure: CREATE PROCEDURE usp_UpdateJobRole
@jobRole VARCHAR(50) AS BEGIN      SET NOCOUNT ON      BEGIN TRAN
BEGIN TRY      UPDATE Employee SET HireDate = GETDATE()
WHERE JobRole = @jobRole      COMMIT TRAN      END TRY
BEGIN CATCH      ROLLBACK TRAN      END CATCH END -- Função:
CREATE FUNCTION ufi_GetLocality (@Code INT) RETURNS VARCHAR(50)
AS BEGIN      DECLARE @Locality VARCHAR(50)      SELECT @Locality =
Locality FROM Location WHERE Code = @Code      IF(@Locality IS
NULL) SET @Locality = 'Unknown'      RETURN @Locality END
```

Transações

- **COMMIT** — transação concluída com sucesso; alterações permanentes
- **ROLLBACK** — algo correu mal; desfaz tudo, BD volta ao estado anterior

Cursors SQL

Permitem aceder a cada tuplo **um de cada vez** (como um apontador). Ciclo: DECLARE → OPEN → FETCH (em loop) → CLOSE → DEALLOCATE

Controlo de Acesso em SQL

- **GRANT** — conceder privilégios
- **REVOKE** — retirar privilégios
- Cada objeto tem um **dono** que controla os privilégios

10. Normalização

? "Objetivos da normalização e impacto no desempenho" ★ (Pergunta 5 do exame 2024/2025)

Objetivo: Analisar uma relação com base na sua chave primária e nas dependências funcionais entre atributos. Minimizar redundância de dados e eliminar anomalias de atualização.

Impacto no desempenho: À medida que a normalização avança, as tabelas são divididas em mais relações → acesso à informação requer mais operações de **junção** → pode afetar negativamente a **performance de leitura/consulta**.

Anomalias de Atualização

Anomalia	Exemplo
Inserção	Inserir um funcionário num escritório que ainda não existe
Remoção	Ao apagar o único funcionário, apaga-se também o escritório
Modificação	Atualizar dados duplicados parcialmente → inconsistência

? Formas Normais ★ (Pergunta 7 do exame 2024/2025 — 3 valores!)

Forma Não Normalizada (UNF)

Uma tabela que contém um ou mais **grupos repetidos**.

1ª Forma Normal (1FN)

Uma relação em que a intersecção entre uma linha e uma coluna contenha **um e um só valor**.

UNF → 1FN: Remover grupos repetidos (achatar a tabela ou criar nova relação com cópia das PKs).

2ª Forma Normal (2FN)

Uma relação na 1FN onde todos os atributos não pertencentes à PK são **totalmente dependentes** de qualquer chave candidata.

1FN → 2FN: Remover **dependências parciais** — atributos que dependem apenas de parte da PK vão para nova tabela.

3ª Forma Normal (3FN)

Uma relação na 2FN onde nenhum atributo não pertencente à PK depende **transitivamente** da PK.

2FN → 3FN: Remover **dependências transitivas** ($A \rightarrow B \rightarrow C$) — criar nova tabela para o determinante transitivo.

Forma Normal Boyce-Codd (FNBC)

Uma relação está na FNBC se e só se todo o **determinante** é chave candidata.



Como funciona a Normalização (Explicação Intuitiva)

Se nunca percebeste normalização, pensa nisto como **organizar o guarda-roupa para não teres coisas repetidas nem misturadas**:

- **UNF (O Caos):** Tens um papel gigante (a Fatura) onde escreveste tudo sobre a empresa, o cliente, e uma lista de 10 artigos comprados. O problema? Se o cliente comprar de novo amanhã, vais escrever o nome, morada e NIF dele tudo outra vez. Se quiseres mudar a morada do cliente, tens de procurar **todas** as faturas dele para mudar.
- **1FN (A Lista Plana):** A base de dados não lida bem com "listas de artigos" dentro de uma fatura. Então, na 1FN, "achatas" tudo. Se a fatura tem 3 artigos, passas a ter 3 linhas separadas para essa fatura na base de dados. Cada linha tem as informações da fatura **TODAS** repetidas + as informações de um dos artigos.
- **2FN (Cortar o que depende só de "metade"):** Reparas que a chave principal agora é composta por (NumFatura + CodArtigo) para identificar cada linha. Mas pensas: "Espera, o **Nome do Artigo** só depende do CodArtigo! Não faz sentido repeti-lo 50 vezes cada vez que ele aparece numa fatura diferente." Então, pegas no que **só depende do artigo** e crias a tabela Artigo. Fazes o mesmo para o que só depende da fatura (criando a tabela Fatura).
 - Resumo 2FN: O que só depende de uma parte da Chave Primária vai para a sua própria tabela.
- **3FN (Tirar os "penduras"):** Olhas para a tua nova tabela Fatura e vês o NIF do Cliente, Nome do Cliente e Morada do Cliente. Tudo depende do NumFatura, certo? Sim, mas... o Nome e Morada do Cliente na verdade dependem do **NIF do Cliente**, e é o NIF que depende da Fatura! Isto é uma dependência "em cadeia" ou **transitiva** (Fatura → NIF Cliente → Nome Cliente). Então, pegas nos "penduras" e crias uma tabela Cliente.
 - Resumo 3FN: Tudo tem de depender APENAS da Chave Primária e de mais nada. Se o atributo B depende do atributo A (e nenhum deles é chave), vão para uma tabela nova!



MÉTODO DO PROFESSOR: Normalização por Atributos (passo a passo)

⚠ O professor quer que identifiques os atributos (com letras), escrevas a chave primária, e mostres as dependências funcionais com setas (→) em cada passo. **NÃO** saltes direto para tabelas!

Passo 0 — Analisar o documento e identificar TODOS os atributos

Olha para a fatura/documento e lista **todos** os dados que encontras. Atribui uma **letra** a cada atributo para simplificar.

Exemplo com a Fatura do exame 2024/2025:

Letra	Atributo
A	NIF_Empresa
B	Nome_Empresa
C	Morada_Empresa
D	CodPostal_Empresa
E	NumFatura
F	Data
G	Hora
H	NIF_Cliente
I	Mesa
J	Empregado
K	CodArtigo
L	Descrição_Artigo
M	Quantidade
N	Preço_Unitário
O	TaxaIVA
P	Subtotal_Linha
Q	Total_Fatura
R	MetodoPagamento
S	ATCUD
T	Incidência_IVA

U	Valor_IVA
---	-----------

Forma Não Normalizada (UNF):

Fatura(A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U)

Definição UNF: Uma tabela que contém um ou mais grupos repetidos.

Passo 1 — Converter para 1ª Forma Normal (1FN)

Definição 1FN: Uma relação em que a intersecção entre uma linha e uma coluna contenha **um e um só valor**.

Remover grupos repetidos: Os artigos (K, L, M, N, O, P) repetem-se em cada fatura → achatar a tabela.

Identificar a Chave Primária:

- Uma fatura (E) tem vários artigos (K)
- **PK = (E, K)**

Fatura_1FN(E, K, A, B, C, D, F, G, H, I, J, L, M, N, O, P, Q, R, S, T, U)
↑PK↑

Identificar TODAS as Dependências Funcionais:

E → A, B, C, D, F, G, H, I, J, Q, R, S (dependem só do n° fatura)
 K → L, N, O (dependem só do código do artigo)
 E, K → M, P (dependem da combinação fatura+artigo)
 A → B, C, D (NIF empresa determina nome, morada, cod.postal)
 O → T, U (taxa IVA determina incidência e valor)

Passo 2 — Converter para 2ª Forma Normal (2FN)

Definição 2FN: Uma relação na 1FN onde todos os atributos não pertencentes à PK são **totalmente dependentes** de qualquer chave candidata.

Identificar Dependências PARCIAIS (atributos que dependem de PARTE da PK):

DEPENDÊNCIAS PARCIAIS IDENTIFICADAS:

C, D, F, G, H, I, J, Q, R, S		E → A, B,
K → L, N, O		← depende só de E (parte da PK)
		←

depende só de K (parte da PK)

DEPENDÊNCIA TOTAL (fica na tabela original):

P		E, K → M,
		← depende da PK completa

✓

Resultado — Separar em tabelas pela decomposição das dependências parciais:

Fatura(E, A, B, C, D, F, G, H, I, J, Q, R, S)	PK: E
Artigo(K, L, N, O)	PK: K
LinhaFatura(E, K, M, P)	PK: (E,

K)

FK:

$E \rightarrow \text{Fatura}$, $K \rightarrow \text{Artigo}$

Agora identificar Dependências TRANSITIVAS nas tabelas da 2FN:

DEPENDÊNCIAS TRANSITIVAS NA TABELA Fatura:

$A \rightarrow B, C, D$	$E \rightarrow A$ e $\leftarrow B, C, D$ dependem de A, não diretamente de E! (transitiva: $E \rightarrow A \rightarrow B, C, D$)
-------------------------	---

DEPENDÊNCIAS TRANSITIVAS NA TABELA Artigo:

$O \rightarrow T, U$	$K \rightarrow O$ e $\leftarrow T, U$ dependem de O, não diretamente de K! (transitiva: $K \rightarrow O \rightarrow T, U$)
----------------------	--

Passo 3 — Converter para 3ª Forma Normal (3FN)

Definição 3FN: Uma relação na 2FN onde nenhum atributo não pertencente à PK depende transitivamente da PK.

Remover as dependências transitivas → criar novas tabelas:

TABELAS FINAIS NA 3FN:

<u>A, B, C, D</u>	PK: A (NIF_Empresa)	Empresa (
F, G, A, H, I, J, Q, R, S)	PK: E (NumFatura)	Fatura (E,
FK: A → Empresa		FK: H →
Cliente (se existir tabela)	Artigo (K, L, N, O)	
PK: K (CodArtigo)		FK: O
→ TaxaIVA	LinhaFatura (E, K, M, P)	PK: (E, K)
FK: E → Fatura		FK: K →
Artigo	TaxaIVA (O, T, U)	PK: O (TaxaIVA)

Escrevendo com os nomes reais dos atributos:

Empresa (NIF_Empresa, Nome_Empresa, Morada, CodPostal)	PK:
NIF_Empresa	Fatura (NumFatura, Data, Hora, NIF_Empresa,
NIF_Cliente, Mesa, Empregado, Total, MetodoPagamento, ATCUD)	
PK: NumFatura	FK: NIF_Empresa → Empresa
Artigo (CodArtigo, Descricao, PrecoUnitario, TaxaIVA)	PK: CodArtigo
FK: TaxaIVA → TaxaIVA	LinhaFatura (NumFatura, CodArtigo, Quantidade, Subtotal)
PK: (NumFatura, CodArtigo)	FK: NumFatura →
Fatura	FK: CodArtigo → Artigo
TaxaIVA (Taxa, Incidencia, ValorIVA)	PK: Taxa



Exemplo 2: Normalização da Pauta (BD-Todas-As-Perguntas)

Passo 0 — Listar todos os atributos da Pauta

Pauta: AnoLetivo, CódigoCurso, Curso, CódigoDocente, Docente, CódigoDisciplina, Disciplina, Ano, CódigoTipoProva, TipoProva,

NºMatricula, NomeAluno, Nota, Obs, DescriçãoObs, Data, HorárioConsulta

Passo 1 — 1FN

Chave Primária:(AnoLetivo, CódigoDisciplina, CódigoTipoProva, NºMatricula, Obs)

Dependências Funcionais identificadas:

DEPENDÊNCIAS PARCIAIS (dependem de PARTE da PK):

CódigoDisciplina → CódigoCurso, Curso, Disciplina, Ano

CódigoTipoProva → TipoProva NºMatricula → NomeAluno Obs →

DescriçãoObs AnoLetivo, CódigoDisciplina → CódigoDocente,

Docente AnoLetivo, CódigoDisciplina, CódigoTipoProva → Data,

HorárioConsulta DEPENDÊNCIA TOTAL (PK completa): AnoLetivo,

CódigoDisciplina, CódigoTipoProva, NºMatricula → Nota

Passo 2 — 2FN (remover dependências parciais)

Disciplina(CódigoDisciplina, Disciplina, CódigoCurso, Curso, Ano)

PK: CódigoDisciplina TipoProva(CódigoTipoProva, TipoProva)

PK: CódigoTipoProva Aluno(NºMatricula, NomeAluno)

PK: NºMatricula Observações(Obs, DescriçãoObs)

PK: Obs Regente(AnoLetivo, CódigoDisciplina, CódigoDocente, Docente)

PK: (AnoLetivo, CódigoDisciplina)

Consulta(AnoLetivo, CódigoDisciplina, CódigoTipoProva, Data,

Horário) PK: (AnoLetivo, CódigoDisciplina, CódigoTipoProva)

Nota(AnoLetivo, CódigoDisciplina, CódigoTipoProva, NºMatricula, Nota)

PK: (AnoLetivo, CódigoDisciplina, CódigoTipoProva,

NºMatricula) Pauta(AnoLetivo, CódigoDisciplina, CódigoTipoProva, NºMatricula, Obs)

PK: (AnoLetivo, CódigoDisciplina,

CódigoTipoProva, NºMatricula, Obs)

Identificar dependências TRANSITIVAS:

Na tabela Disciplina: CódigoDisciplina → CódigoCurso → Curso

Na tabela Regente: (AnoLetivo, CódigoDisciplina) → CódigoDocente

→ Docente

Passo 3 — 3FN (remover dependências transitivas)

NOVAS TABELAS criadas: Curso(CódigoCurso, Curso) PK:

CódigoCurso Docente(CódigoDocente, Docente) PK:

CódigoDocente TABELAS ALTERADAS (removem-se os atributos transitivos):

Disciplina(CódigoDisciplina, Disciplina, CódigoCurso, Ano)

FK: CódigoCurso → Curso

Regente(AnoLetivo, CódigoDisciplina, CódigoDocente) FK:

CódigoDocente → Docente



Resumo do Método (cheat sheet para o exame)

1. IDENTIFICAR ATRIBUTOS do documento (atribuir letras)

2. UNF: escrever TODOS os atributos numa só relação

3. 1FN: Definição + Identificar a CHAVE PRIMÁRIA

- + Listar TODAS as dependências funcionais
- 4. 2FN: Definição + Identificar DEPENDÊNCIAS PARCIAIS (atributos que dependem de PARTE da PK)
 - Separar em novas tabelas
 - + Identificar DEPENDÊNCIAS TRANSITIVAS
- 5. 3FN: Definição + Remover DEPENDÊNCIAS TRANSITIVAS ($A \rightarrow B \rightarrow C$, onde C depende de B e não de A)
 - Criar novas tabelas para os determinantes
- 6. Escrever tabelas finais com PK e FK identificadas

└─ DICA: Em cada forma normal, **ESCREVE SEMPRE A DEFINIÇÃO** antes de aplicar! O professor pede explicitamente: "Enuncie as definições de cada Forma Normal à medida que faz a normalização"

Desnormalização

Processo de **adicionar redundância** para otimizar performance de leitura. Exemplo: tabela de publicações de blog (escrita uma vez, lida constantemente).

11. Desenho e Modelação de BD (Diagramas E/R)

? "Passos do Desenho Conceptual e Lógico"

Desenho Conceptual:

1. Identificar tipos de entidades
2. Identificar tipos de relacionamentos
3. Identificar e associar atributos
4. Determinar domínios dos atributos
5. Determinar chaves primárias e candidatas
6. Considerar modelação avançada (opcional)
7. Verificar redundância
8. Validar com transações dos utilizadores
9. Rever com o utilizador

Desenho Lógico:

1. Remover componentes não compatíveis com modelo relacional (opcional)
2. Obter as relações para o modelo lógico
3. Validar relações usando normalização
4. Validar com transações de utilizadores

5. Definir restrições de integridade
6. Rever com o utilizador

Tipos de Atributos (Diagrama E/R)

Tipo	Exemplo
Simples	Nº Cartão de Cidadão
Composto	Endereço → (morada, cidade, código postal)
Multi-valor	Grau académico → (licenciado, mestre, doutorado)
Derivado	Idade (calculada a partir da data de nascimento)

? "Especialização vs Generalização"

Especialização	Generalização
Definir subclasses de uma superclasse	Identificar características comuns → criar superclasse
De cima para baixo (top-down)	De baixo para cima (bottom-up)
Ex: EMPREGADO → Secretária, Engenheiro, Técnico	Ex: CARRO + CAMIÃO → VEÍCULO

Ciclo de Vida de uma Aplicação de BD

1. Planeamento da BD
2. Definição do Sistema
3. Recolha e Análise de Requisitos
4. Desenho da BD
5. Seleção do SGBD (opcional)
6. Desenho da Aplicação
7. Prototipagem (opcional)
8. Implementação
9. Conversão e Alimentação de Dados
10. Testes

12. Data Warehousing

? "Benefícios e problemas dos Data Warehouses" ★ (Pergunta 6 do exame 2024/2025)

Definição (Inmon, 1993): Coleção de dados orientada a assuntos, integrada, variável no tempo e não-volátil em suporte ao processo de tomada de decisão.

Benefícios

- Grande potencial de retorno sobre investimento (ROI)
- Vantagem competitiva
- Incremento de produtividade dos decision-makers

Problemas

- Subestimação dos recursos necessários ao carregamento
- Problemas escondidos nos sistemas fonte
- Dados necessários não capturados
- Crescimento dos pedidos dos utilizadores
- Homogeneização dos dados
- Necessita de grandes recursos
- Dados proprietários
- Manutenção elevada
- Projetos de longa duração
- Complexidade da integração

5 Fluxos de Dados

Fluxo	Processos
Inflow	Extração, limpeza e carregamento (ETL)
Upflow	Sumarização, agregação e distribuição
Downflow	Arquivamento e backup
Outflow	Disponibilização aos utilizadores finais
Metaflow	Gestão dos metadados

Componentes da Arquitetura

- Operational Data / Operational Data Store
- ETL Manager (Load Manager)
- Warehouse Manager
- Query Manager
- Detailed Data / Summarized Data
- Archive/Backup
- Metadata
- End-User Access Tools

? "Data Mart vs Data Warehouse"

Data Mart	Data Warehouse
Subconjunto do DW	Coleção completa de dados
Para um departamento/função	Para toda a organização
Mais simples e barato	Mais complexo e caro
Utilizadores facilmente definidos	Mais abrangente

13. BD Distribuídas e Paralelas

4 Estratégias de Alocação de Dados

Estratégia	Descrição
Centralizada	Uma só BD e SGBD num site, utilizadores distribuídos por rede
Particionada	BD particionada em fragmentos independentes, cada um num site
Replicação Completa	Cópia completa da BD em cada site
Replicação Seletiva	Combinação de particionamento, replicação e centralização

Fragmentação

Razões: Uso (aplicações usam vistas, não relações completas), Eficiência (dados perto de onde são usados), Paralelismo, Segurança.

Desvantagens: Performance e integridade.

Arquiteturas de SGBDs Paralelos

- **Memória Partilhada** — processadores partilham memória comum
- **Disco Partilhado** — processadores partilham disco comum
- **Nada Partilhado** — sem partilha de memória nem disco

14. Exercícios Tipo Exame (com resolução)

Os exercícios foram movidos para um documento separado para facilitar o estudo. 🖱️

Abrir Documento de Exercícios (Exercicios_Exames_BD.md)

Análise de Frequência — Perguntas que Saíram nos 3 Exames

Tema	2020/2021	2022/2023	2024/2025	Frequência
Integridade Referencial (ON DELETE/UPDATE)	✓	✓	✓	3/3 ●
Normalização de Fatura	✓	✓	✓	3/3 ●
SQL prático (SELECT + JOIN + GROUP BY)	✓	✓	✓	3/3 ●
Álgebra Relacional prática	✓	✓	✓	3/3 ●
Vistas (Views)	✓	—	✓	2/3 ●
Triggers	✓	—	✓	2/3 ●
Independência de Dados / Arq. ANSI/SPARC	—	✓	—	1/3 ●

Cliente-Servidor 2 vs 3 níveis	—	✓	—	1/3 ●
Subquery vs Junção	—	✓	—	1/3 ●
Anomalias de atualização	✓	—	—	1/3 ●
Tipos de Join (Natural, Theta, Outer)	✓	—	—	1/3 ●
Atributos em diag. E/R	—	✓	—	1/3 ●
Abordagens múltiplas vistas	—	✓	—	1/3 ●
LMD Procedimental vs Não-Procedimental	—	—	✓	1/3 ●
Normalização (teoria + objetivos)	—	—	✓	1/3 ●
Data Warehouses	—	—	✓	1/3 ●
Metodologia de desenvolvimento BD	✓	—	—	1/3 ●
Diag. E/R + tabela associativa	—	✓	✓	2/3 ●



Resumo Rápido — O que Estudar por Prioridade



Prioridade MÁXIMA (saiu em TODOS os exames):

1. **Integridade Referencial** — ON DELETE / ON UPDATE (CASCADE, SET NULL, SET DEFAULT, NO ACTION)

2. **Normalização de Fatura** — UNF → 1FN → 2FN → 3FN com dependências funcionais (3 val.!)
3. **SQL prático** — SELECT com JOIN, GROUP BY, HAVING, subqueries
4. **Álgebra Relacional prática** — seleção, projeção, junção, diferença
5. **Diagrama E/R + Tabela Associativa** — identificar PK, FK, atributos

● **Prioridade Alta (saiu em 2 de 3 exames):**

1. **Vistas (Views)** — definição, diferenças de relação base, materialização
2. **Triggers** — definição, tipos (BEFORE/AFTER/INSTEAD OF), vantagens/desvantagens

● **Prioridade Média (saiu em 1 de 3 exames — pode calhar!):**

- LMD Procedimental vs Não-Procedimental
- Independência de dados / Arquitetura ANSI/SPARC
- Cliente-Servidor 2 vs 3 níveis
- Subquery vs Junção
- Anomalias de atualização
- Tipos de Join
- Atributos em diagramas E/R
- Data Warehouses
- Metodologia de desenvolvimento de BD
- Abordagens para múltiplas vistas de utilizadores

● **Complementar (nunca saiu nos 3 exames analisados, mas está na matéria):**

- Conceitos fundamentais (BD, SGBD, Metadados, System Catalog)
- BD Distribuídas e Paralelas
- Cursores SQL
- SGBD Orientados a Objetos
- Stored Procedures vs Funções
- Controlo de concorrência
- Transações (COMMIT/ROLLBACK)



Dicas para o Exame

1. **Normalização vale SEMPRE 3 valores** — saiu nos 3 exames analisados! Pratica com faturas reais

2. **SQL + Álgebra Relacional valem 4-5 valores** — treina queries com JOIN, GROUP BY, HAVING
3. **Integridade Referencial sai SEMPRE** — memoriza as 4 ações (CASCADE, SET NULL, SET DEFAULT, NO ACTION)
4. **Nas perguntas teóricas:** dá definições claras + exemplos sempre que possível
5. **Tempo:** 2h para 8 perguntas $\approx$ 15 min/pergunta; a normalização e SQL/ÁR precisam de mais tempo
6. **Sem consulta** — memoriza as definições das Formas Normais e sabe fazer o processo passo a passo
7. **Padrão do exame:** 6 perguntas teóricas (2 val. cada = 12 val.) + normalização (3 val.) + SQL/ÁR (5 val.)
8. **Álgebra Relacional com DIFERENÇA (-)** apareceu em todos os exames — domina o padrão "quais os X que NÃO..."

💡 **Nota:** Este guia foi gerado a partir dos slides PPS (Aula 1–11), do documento "BD-Todas-As-Perguntas", dos helpers (normalização, T-SQL, fatura), e dos exames de **2020/2021**, **2022/2023** e **2024/2025** como referência.